

```

import { goldPlus, probe, schaden } from
"./engine";
import { vielleichtHeiltrank } from "./heal";
import type { Runtime } from "./runtime";
import { LEICHT, MITTEL, SCHWER, tot, type Held
} from "./types";

export async function dorfMuehle(rt: Runtime,
held: Held) {
  if (held.loesungswegMuehle) {
    await muehleNachspiel(rt, held);
    return;
  }

  if (!held.muehleBesucht) {
    held.muehleBesucht = true;
    await rt.present({
      title: "Die stumme Mühle",
      art: "mill",
      portrait: null,
      held,
      lines: [
        "Kein Mehlstaub in der Luft, obwohl das
Rad sich dreht.",
        "Vor der Tür lehnt ein leerer Karren,
dessen Deichsel schon Moos angesetzt hat.",
        "Bertok steht im Eingang, bevor du
klopfen kannst. Seine Hände sind mehlweiß,
obwohl seit Tagen nichts gemahlen wurde.",
        "Hinter ihm bewegt sich etwas zwischen
den Säcken – zu schnell für eine Ratte.",
      ],
    });
  }
}

```

```

while (!tot(held) && !held.loesungswegMuehle)
{
    const uferLabel = held.spurenGefunden
        ? "Den Schleifspuren zum Lagerhaus folgen"
        : held.muellerVertraut
            ? "Dem Mann nachgehen, der zu oft am
Ufer steht"
            : "Dem Uferweg flussabwärts folgen";
    const items: { id: string; label: string }[]
= [
    { id: "bertok", label: "Mit Bertok am
Mahlwerk sprechen" },
    { id: "lene", label: "Zu Lene in die
Kornkammer gehen" },
    { id: "rad", label: "Das Wasserrad und den
Uferweg ansehen" },
    { id: "ufer", label: uferLabel },
    ];
    if (held.fluechtlingeEntdeckt) {
        items.push({ id: "verrat", label: "Holm
die versteckte Familie nennen" });
    }
    items.push({ id: "dorf", label: "Die Mühle
verlassen" });

    const wahl = await rt.present({
        title: "Mühle",
        art: "mill",
        portrait: null,
        held,
        lines: [
            "Das Rad schlägt gegen das Wasser und
mahlt nichts.",
            "Bertok bleibt am Mahlwerk. Aus der
Kornkammer kommt das leise Klicken von gezählten
Säcken.",

```

```

        held.spurenGefunden
        ? "Unten am Ufer sind die nassen
Schleifspuren noch immer im Schlamm."
        : "Hinter dem Haus fällt der Boden zum
Fluss ab.",
    ],
    choices: items.map((item) => item.label),
});
const id = items[wahl]?.id;
if (id === "bertok") await muehleBertok(rt,
held);
    else if (id === "lene") await muehleLene(rt,
held);
    else if (id === "rad") await muehleRad(rt,
held);
    else if (id === "ufer") await muehleUfer(rt,
held);
    else if (id === "verrat") await
muehleVerrat(rt, held);
    else return;
}
}

async function muehleNachspiel(rt: Runtime,
held: Held) {
    if (held.loesungswegMuehle === "verraten") {
        await rt.present({
            title: "Mühle",
            art: "mill",
            portrait: null,
            held,
            lines: [
                "Das Rad mahlt wieder. Der Schutzbrief
hängt schief am Türsturz.",
                "Bertok grüßt dich mit dem Kopf, nicht
mit der Hand.",
                "Die Kornkammer ist aufgeräumt. Zu

```

```

    aufgeräumt.",
    ],
  });
  return;
}
if (held.loesungswegMuehle === "kampf") {
  await rt.present({
    title: "Mühle",
    art: "mill",
    portrait: null,
    held,
    lines: [
      "Mehlstaub steht wieder in der Luft.",
      "Bertok bedankt sich knapp und schließt
die Tür einen Spalt früher als nötig.",
      "Am Steg klebt noch etwas Dunkles am
Holz. Der Regen holt es nicht ganz runter.",
    ],
  });
  return;
}
await rt.present({
  title: "Mühle",
  art: "mill",
  portrait: "miller",
  held,
  lines: [
    "Das Rad dreht sich lauter als sonst.",
    "Bertok reicht dir nichts mehr. Lene zählt
keine Säcke.",
    "Manche Schulden werden nicht bezahlt. Sie
werden nur nicht mehr eingetrieben.",
  ],
});
}

async function muehleBertok(rt: Runtime, held:

```

```

Held) {
    const lines = [
        "Bertok prüft das Mahlwerk, obwohl es längst
justiert ist. Der Stein ist kalt.",
        "„Kein Mehl heute. Kein Mehl seit zwei
Wochen.“ Er sagt es, bevor du fragst.",
    ];
    if (held.muellerVertraut) {
        lines.push("Seine Stimme bleibt leise. Er
sieht zur Kornkammer, dann zum Ufer, als könnte
beides ihn hören.");
    } else if (held.bertokBedraengt) {
        lines.push("Seit du Druck gemacht hast,
bleiben seine Sätze kurz. Die Hand auf dem
Mahlstein zittert trotzdem.");
    } else {
        lines.push("„Das Wasser steht zu niedrig.
Das Korn ist schlecht. Ärger mit der Lieferung.“
Namen nennt er nicht.");
    }

    const items: { id: string; label: string }[] =
[];
    if (!held.muellerVertraut &&
!held.bertokBedraengt) {
        items.push({ id: "vertrauen", label:
"Vertrauen gewinnen (Charisma, mittel)" });
        items.push({ id: "druck", label: "Druck
machen" });
    } else if (held.muellerVertraut &&
!held.renniksBeweis) {
        items.push({ id: "schuld", label: "Nach der
Schuld fragen, die ihn drückt" });
    } else if (held.muellerVertraut) {
        items.push({ id: "ufermann", label: "Noch
einmal nach dem Mann am Ufer fragen" });
    }
}

```

```

    items.push({ id: "zurueck", label: "Zurück in
die Mühle" });

    const wahl = await rt.present({
      title: "Bertok am Mahlwerk",
      art: "mill",
      portrait: null,
      held,
      lines,
      choices: items.map((item) => item.label),
    });
    const id = items[wahl]?.id;

    if (id === "vertrauen") {
      const ergebnis = probe(held, "Charisma",
held.charisma, MITTEL, "Bertoks Vertrauen");
      if (ergebnis.erfolg) {
        held.muellerVertraut = true;
        await rt.present({
          held,
          probe: ergebnis,
          lines: [
            "Bertok lässt die Hand vom Stein. „Ein
Mann steht zu oft am Ufer. Er wiegt Getreide,
das nicht seines ist.“",
            "Er nennt keinen Namen. Der Blick zum
Fluss genügt.",
            "„Wenn du gehst, geh nicht laut. Er
hat einen Wächter, der auf Geräusche wartet,
nicht auf Gründe.“",
          ],
        });
      } else {
        await rt.present({
          held,
          probe: ergebnis,
          lines: [

```

```

        "Bertok wiederholt die Ausrede.
Schlechtes Korn. Niedriges Wasser.",
        "Die Hand auf dem Mahlstein zittert.
Mehr gibt er nicht her.",
    ],
    });
}
return;
}

if (id === "druck") {
    held.bertokBedraengt = true;
    await rt.present({
        held,
        lines: [
            "Du trittst näher, bis Bertok den Stein
zwischen euch braucht.",
            "„Rennik“, sagt er schließlich. Der Name
fällt wie ein Werkzeug, das man fallen lässt,
weil es zu heiß ist.",
            "Aus der Kornkammer wird es still. Lene
hat mitgehört.",
            "Bertok warnt dich nicht vor dem
Wächter. Er hat schon zu viel gesagt.",
        ],
    });
    return;
}

if (id === "schuld") {
    held.renniksBeweis = true;
    await rt.present({
        title: "Bertok am Mahlwerk",
        art: "evidence",
        portrait: null,
        held,
        lines: [

```

```

        "Bertok zieht aus dem Mahlstein einen
        gefalteten Schein, so flach, als hätte ihn das
        Gewicht dort versteckt.",
        "Renniks Hand. Bertoks Zeichen. Darunter
        eine zweite Schuld, die Rennik selbst in einem
        Nachbarort offen hat.",
        "„Wenn du das zeigst, bin ich ihn los.
        Oder ich bin alles los.“",
        ],
    });
    return;
}

if (id === "ufermann") {
    await rt.present({
        held,
        lines: [
            "„Lagerhaus flussabwärts. Halb verhängte
            Fenster. Er wiegt dort ab, was er hier nicht
            mahlen lässt.“",
            held.rennikGewarnt
            ? "Bertok schluckt. „Wenn er dich
            schon gehört hat, nützt mein Name nichts mehr.“"
            : "„Sag, Bertok schickt jemanden, der
            Fragen stellt. Mehr Kredit habe ich dort
            nicht.“",
        ],
    });
}

async function muehleLene(rt: Runtime, held:
Held) {
    held.sennaBesuche += 1;
    const still = held.bertokBedraengt ||
held.leneBedraengt;

```

```

    if (held.fluechtlingeEntdeckt) {
      await rt.present({
        title: "Kornkammer",
        art: "mill",
        portrait: "miller",
        held,
        lines: [
          "Lene steht vor der Nische, als könnte
          ihr Körper eine Tür ersetzen.",
          "Hinter ihr hält ein Mann ein
          zerbrochenes Hofwerkzeug fest. Zwei Kinder atmen
          zu gleichmäßig.",
          "„Du hast sie gesehen“, sagt Lene. „Dann
          entscheide, ob das Dorf sie auch sehen soll.“",
        ],
      });
      return;
    }

    const lines = [
      "Lene zählt dieselben Mehlsäcke immer wieder
      nach. Die Zahlen ändern sich nicht.",
      "Ihre Finger bleiben an einem Sack hängen,
      der hohl klingt.",
    ];
    if (still) {
      lines.push("Sie hat Bertoks Stimme gehört.
      Seither redet sie, als läge in jedem Satz ein
      Preis.");
    } else if (held.mehlsackGefunden ||
      held.mehlsackGemeldet) {
      lines.push("Du erkennst die Frau vom
      Brunnen. Die Müllerin. Hier heißt sie Lene, und
      sie zählt, als könnte Zählen eine Tür
      verschließen.");
    } else {
      lines.push("Am Brunnen nennt man sie die

```

```

Müllerin. Hier ist sie nur jemand, der nicht
will, dass du die hintere Wand ansiehst.");
}

const items: { id: string; label: string }[] =
[ { id: "warten", label: "Warten und zuhören" } ];
if (!still) items.push({ id: "suchen", label:
"Die Kammer durchsuchen (Geschick, mittel)" });
items.push({ id: "druck", label: "Sie
bedrängen" }, { id: "zurueck", label: "Zurück
zum Mahlwerk" });

const wahl = await rt.present({
  title: "Lene in der Kornkammer",
  art: "mill",
  portrait: "miller",
  held,
  lines,
  choices: items.map((item) => item.label),
});
const id = items[wahl]?.id;

if (id === "warten") {
  if (still) {
    await rt.present({
      held,
      lines: [
        "Lene zählt weiter. Zwölf. Zwölf.
Zwölf.",
        "„Nicht heute“, sagt sie, ohne dich
anzusehen.",
      ],
    });
    return;
  }
  if (held.sennaBesuche >= 2) {
    await rt.present({

```

```

        held,
        lines: [
            "Beim zweiten Mal bleibt Lene mit der
Hand auf einem Sack stehen.",
            "„Bertok hat eine Schuld“, sagt sie
beiläufig, als wäre das Wetter. „Schulden haben
Hände. Manche stehen am Ufer.“",
            "Mehr gibt sie nicht. Aber du weißt
jetzt, dass die Mühle nicht wegen des Wassers
stillsteht.",
        ],
    });
    return;
}
await rt.present({
    held,
    lines: [
        "Du bleibst, bis das Zählen langsamer
wird.",
        "Lene sagt nichts, das man vor einem
Fremden sagen dürfte. Ihre Schulter bleibt zur
hinteren Wand gedreht.",
    ],
    });
    return;
}

if (id === "suchen") {
    const schwer = held.spurenGefunden ||
held.sennaBesuche >= 2 ? MITTEL : SCHWER;
    const ergebnis = probe(held,
"Geschicklichkeit", held.geschick, schwer, "die
Kornkammer prüfen");
    if (ergebnis.erfolg) {
        held.fluechtlingeEntdeckt = true;
        await yorwinFund(rt, held, ergebnis);
    } else {

```

```

        await rt.present({
            held,
            probe: ergebnis,
            lines: [
                "Du schiebst Säcke beiseite, findest
Mehl und eine Wand, die hohl klingt, sonst
nichts.",
                "Lene stellt den Sack zurück, mit dem
Rücken zur Wand. „Du suchst am falschen Ort. Geh
jetzt.“",
            ],
        });
    }
    return;
}

if (id === "druck") {
    held.leneBedraengt = true;
    if (held.sennaBesuche >= 2 && !still) {
        await rt.present({
            held,
            lines: [
                "Lene weicht bis an die hintere Wand
zurück.",
                "„Die Schuld“, sagt sie schnell.
„Rennik. Ufer. Mehr nicht.“",
                "Danach zählt sie wieder. Diesmal ohne
dich anzusehen.",
            ],
        });
        return;
    }
    await rt.present({
        held,
        lines: [
            "Lene schüttelt den Kopf, bevor du den
Satz zu Ende hast.",

```

```

        "Hinter der Wand wird ein Kind stiller,
als es vorher war. Du tust, als hättest du den
Unterschied nicht gehört.",
        ],
    });
}

}

async function yorwinFund(rt: Runtime, held:
Held, ergebnis: ReturnType<typeof probe>) {
    await rt.present({
        title: "Hinter der Nische",
        art: "mill",
        portrait: "miller",
        held,
        probe: ergebnis,
        lines: [
            "Die hintere Wand gibt nach. Dahinter
sitzt ein Mann mit einem zerbrochenen
Hofwerkzeug in der Faust.",
            "Zwei Kinder halten sich an seinem Ärmel.
Lene stellt sich davor, zu spät.",
            "„Yorwin“, sagt sie. „Meine Schwester ist
tot. Die Kinder nicht. Rennik weiß das. Deshalb
mahlt hier nichts.“",
            "Yorwin sieht nicht dich an, sondern die
Tür. „Wenn das Dorf uns findet, zahlt Bertok mit
uns.“",
        ],
    });
}

}

async function muehleRad(rt: Runtime, held:
Held) {
    if (held.spurenGefunden) {
        await rt.present({
            title: "Wasserrad",

```

```

        art: "mill",
        portrait: null,
        held,
        lines: [
            "Die Schleifspuren sind noch da. Nass,
            breit, von einem gezogenen Sack.",
            "Daneben der Abdruck eines Kinderschuhs,
            zu klein für Arbeit, zu deutlich für Zufall.",
        ],
    });
    return;
}

const ergebnis = probe(held,
"Geschicklichkeit", held.geschick, LEICHT,
"Spuren am Wasserrad lesen");
if (ergebnis.erfolg) {
    held.spurenGefunden = true;
    await rt.present({
        title: "Wasserrad",
        art: "ditch",
        portrait: null,
        held,
        probe: ergebnis,
        lines: [
            "Unter dem Rad ist das Wasser nicht zu
            niedrig. Es läuft, als hätte niemand je das
            Gegenteil behauptet.",
            "Nasse Schleifspuren führen vom
            Mühlkeller den Uferweg hinab. Keine Tierspur.
            Ein gezogener Sack. Daneben ein Kinderschuh.",
            "Bertoks Ausrede bleibt hinter dir im
            Radgeräusch zurück.",
        ],
    });
    return;
}

```

```

    await rt.present({
      title: "Wasserrad",
      art: "mill",
      portrait: null,
      held,
      probe: ergebnis,
      lines: [
        "Der Uferweg ist nass. Mehr sagst du dir nicht.",
        "Ob jemand dort geht, oder nur das Rad das Ufer schlägt, bleibt unklar.",
        "Wer ohne Spur folgt, kommt später an und lauter.",
      ],
    });
  }

```

```

async function muehleUfer(rt: Runtime, held: Held) {
  const stegSchwer = held.verwundet ? SCHWER : MITTEL;
  await rt.present({
    title: "Uferpfad",
    art: "ditch",
    portrait: null,
    held,
    lines: held.spurenGefunden
      ? [
        "Die Schleifspuren halten bis zu einem morschen Steg.",
        "Dahinter steht ein Lagerhaus mit halb verhängten Fenstern. Innen brennt Licht, das kein Müller braucht.",
      ]
      : [
        "Ohne klare Spur dauert der Weg

```

```

länger. Dornen. Nasser Lehm. Zweimal derselbe
Knick im Ufer.",
    "Schließlich ein morscher Steg und ein
Haus, das leer wirken will und es nicht
schafft.",
    ],
    });

const steg = await rt.present({
    title: "Morscher Steg",
    art: "ditch",
    portrait: null,
    held,
    lines: [
        "Die Bretter geben nach, wo das Wasser sie
am längsten hat.",
        "Auf der anderen Seite lehnt ein Mann
unter dem Vordach, als würde er auf Lieferungen
warten, die nachts kommen.",
    ],
    choices: [
        held.verwundet
        ? "Den Steg überqueren (Stärke, schwer –
du bist verwundet)"
        : "Den Steg überqueren (Stärke,
mittel)",
        "Umkehren zur Mühle",
    ],
    });
if (steg === 1) return;

const stegProbe = probe(held, "Stärke",
held.staerke, stegSchwer, "den morschen Steg
überqueren");
if (!stegProbe.erfolg) {
    const wunde = schaden(held, 3, "Sturz ins
kalte Wasser");

```

```

    held.rennikGewarnt = true;
    held.todesort = tot(held) ? "steg" :
held.todesort;
    await rt.present({
      art: tot(held) ? "death" : "ditch",
      held,
      probe: stegProbe,
      log: [wunde],
      lines: tot(held)
      ? [
        "Das Brett bricht. Das Wasser ist
schneller als der Schrei.",
        "Der Fluss nimmt, was er bekommt. In
der Mühle dreht sich das Rad weiter, für
niemanden im Besonderen.",
      ]
      : [
        "Das Brett bricht. Du gehst bis zur
Hüfte ins Wasser. Die Kälte beißt sofort.",
        "Am Ufer hat der Mann den Kopf
gehoben. Du kommst nicht mehr ungesehen.",
      ],
    });
    if (tot(held)) return;
    await vielleichtHeiltrank(rt, held);
    if (tot(held)) return;
  } else {
    await rt.present({
      held,
      probe: stegProbe,
      lines: [
        "Das Holz hält, weil du es nicht in der
Mitte betrittst.",
        "Der Mann unter dem Vordach rührt sich
noch nicht.",
      ],
    });
  }
}

```

```

    }

    const sneakSchwer = held.rennikGewarnt ?
SCHWER : MITTEL;
    const waechter: { id: string; label: string }
[] = [
    {
        id: "schleich",
        label: held.rennikGewarnt
            ? "Am Wächter vorbeischleichen
(Geschick, schwer)"
            : "Am Wächter vorbeischleichen
(Geschick, mittel)",
    },
];
    if (held.muellerVertraut &&
!held.rennikGewarnt) {
        waechter.push({ id: "name", label: "Sich auf
Bertoks Namen berufen (Charisma, leicht)" });
    }
    waechter.push({ id: "kampf", label: "Den
Wächter niederschlagen (Stärke, mittel)" });
    waechter.push({ id: "zurueck", label: "Zum
Steg zurück" });

    const wahl = await rt.present({
        title: "Lagerhaus am Fluss",
        art: "camp",
        portrait: null,
        held,
        lines: [
            "Der Wächter trägt kein Wappen. Nur nasse
Stiefel und einen Knüppel, der schon benutzt
wurde.",
            held.rennikGewarnt
                ? "Er hat dich im Wasser gesehen. Die
Schulter ist schon zur Tür gedreht."

```

```

        : "Er sieht auf den Fluss, nicht auf den
Steg. Noch.",
    ],
    choices: waechter.map((item) => item.label),
});
const id = waechter[wahl]?.id;
if (id === "zurueck" || id == null) return;

if (id === "name") {
    const ergebnis = probe(held, "Charisma",
held.charisma, LEICHT, "Bertoks Namen nennen");
    if (ergebnis.erfolg) {
        await rt.present({
            held,
            probe: ergebnis,
            lines: [
                "„Bertok schickt jemanden, der Fragen
stellt.“ Der Wächter spuckt, tritt aber zur
Seite.",
                "„Dann soll Rennik selbst hören, wie
dünn dieser Kredit geworden ist.“",
            ],
        });
        await rennikKontor(rt, held, "offen");
        return;
    }
    held.rennikGewarnt = true;
    await rt.present({
        held,
        probe: ergebnis,
        lines: ["Der Name macht ihn nicht
freundlicher. Er hebt den Knüppel.", "„Bertok
hat hier nichts mehr zu schicken.“"],
    });
}

if (id === "schleich") {

```

```

    const ergebnis = probe(held,
"Geschicklichkeit", held.geschick, sneakSchwer,
"am Wächter vorbeischleichen");
    if (ergebnis.erfolg) {
        await rt.present({
            art: "sneak",
            held,
            probe: ergebnis,
            lines: [
                "Du gehst dort, wo das Holz nicht
knarrt.",
                "Der Wächter bleibt am Fluss. Innen
riecht es nach nassem Korn und fremder Tinte.",
            ],
        });
        await rennikKontor(rt, held, "still");
        return;
    }
    held.rennikGewarnt = true;
    const wunde = schaden(held, 2, "Knüppel am
Rippenbogen");
    held.todesort = tot(held) ? "rennik" :
held.todesort;
    await rt.present({
        art: tot(held) ? "death" : "combat",
        held,
        probe: ergebnis,
        log: [wunde],
        lines: tot(held)
            ? ["Der Knüppel findet die Schläfe. Das
Lagerhaus behält, was hereinkommt."]
            : ["Er hört dich. Der Knüppel trifft
zuerst. Danach musst du dich entscheiden, ob du
noch reden willst."],
    });
    if (tot(held)) return;
    await vielleichtHeiltrank(rt, held);

```

```

    if (tot(held)) return;
    await rennikKontor(rt, held, "offen");
    return;
}

const kampf = probe(held, "Stärke",
held.staerke, MITTEL, "den Wächter
niederschlagen");
if (kampf.erfolg) {
    await rt.present({
        art: "combat",
        held,
        probe: kampf,
        lines: [
            "Der Knüppel kommt zu hoch. Du kommst zu
nah.",
            "Der Wächter geht gegen die Tür und
bleibt dort. Innen scharrt ein Stuhl.",
        ],
    });
    await rennikKontor(rt, held, "kampf");
    return;
}
const wunde = schaden(held, 4, "Schlagwechsel
am Steg");
held.todesort = tot(held) ? "rennik" :
held.todesort;
await rt.present({
    art: tot(held) ? "death" : "combat",
    held,
    probe: kampf,
    log: [wunde],
    lines: tot(held)
    ? [
        "Der Wächter schlägt, bis nichts mehr
antwortet.",
        "Der Fluss nimmt später, was das Ufer

```

```

nicht behalten will.",
    ]
    : ["Du bleibst stehen. Der Wächter auch,
mit blutigem Mund. Die Tür ist trotzdem
offen."],
    });
    if (tot(held)) return;
    await vielleichtHeiltrank(rt, held);
    if (tot(held)) return;
    await rennikKontor(rt, held, "kampf");
}

async function rennikKontor(rt: Runtime, held:
Held, ankunft: "still" | "offen" | "kampf") {
    await rt.present({
        title: "Renniks Kontor",
        art: "evidence",
        portrait: null,
        held,
        lines: [
            "Rennik sitzt an einem Tisch, der zu gut
für dieses Haus ist, und wiegt Getreide auf
einer Waage, die ihm nicht gehört.",
            "An der Wand ein Bündel Schuldscheine.
Einer trägt Bertoks Zeichen, schief und zu oft
gefaltet.",
            "Er sieht dich an wie eine Lieferung, die
zu früh gekommen ist.",
            "„Die Mühle mahlt, wenn ich es erlaube.
Das Dorf isst, wenn ich es erlaube. Du stehst
dazwischen.“",
        ],
    });

    const items: { id: string; label: string }[] =
[
    { id: "kampf", label: "Rennik und den

```

```

Wächter vertreiben (Stärke, mittel)" },
];
if (ankunft === "still" ||
held.spurenGefunden) {
  items.push({ id: "schleich", label: "Den
Schuldschein stehlen (Geschick, mittel)" });
}
if (held.renniksBeweis) {
  items.push({ id: "reden", label: "Rennik mit
seiner eigenen Schuld konfrontieren (Charisma,
mittel)" });
}
items.push({ id: "gehen", label: "Ohne
Entscheidung zurück" });

const wahl = await rt.present({
  title: "Renniks Kontor",
  art: "evidence",
  portrait: null,
  held,
  lines: ["Was tust du?"],
  choices: items.map((item) => item.label),
});
const id = items[wahl]?.id;
if (id === "gehen" || id == null) return;

if (id === "schleich") {
  const ergebnis = probe(held,
"Geschicklichkeit", held.geschick, MITTEL,
"Renniks Schuldschein stehlen");
  if (ergebnis.erfolg) {
    held.renniksBeweis = true;
    held.loesungswegMuehle = "schleich";
    held.fluechtlingeEntdeckt = true;
    await rt.present({
      art: "sneak",
      held,

```

```

        probe: ergebnis,
        lines: [
            "Der Schein löst sich vom Nagel, ohne
dass die Waage zuckt.",
            "Rennik redet weiter mit dem Getreide.
Du bist schon wieder am Steg, bevor er merkt,
dass ihm etwas fehlt, das er selbst gestohlen
hat.",
        ],
    });
    await muehleEnde(rt, held);
    return;
}
const wunde = schaden(held, 2, "erwischt am
Kontortisch");
held.todesort = tot(held) ? "rennik" :
held.todesort;
await rt.present({
    held,
    probe: ergebnis,
    log: [wunde],
    lines: tot(held)
        ? ["Rennik hebt nicht selbst die Hand.
Der Wächter schon."]
        : ["Die Waage schlägt an. Rennik hebt
den Kopf. Stehlen ist vorbei."],
    });
    if (tot(held)) return;
    return;
}

if (id === "reden") {
    const ergebnis = probe(held, "Charisma",
held.charisma, MITTEL, "Rennik mit seiner Schuld
stellen");
    if (ergebnis.erfolg) {
        held.loesungswegMuehle = "verhandelt";
    }
}

```

```

    held.fluechtlingeEntdeckt = true;
    await rt.present({
      held,
      probe: ergebnis,
      lines: [
        "Du legst seine zweite Schuld auf die
        Waage. Die Schale senkt sich, als wäre Papier
        schwerer als Korn.",
        "Rennik wird blass unter dem Schmutz.
        „Wenn das im Nachbarort ankommt, wiegt man mich
        wie das Korn, das ich schulde.“",
        "Er nimmt die Hände von Bertoks
        Schein. „Die Mühle ist frei. Die Familie auch.
        Ich war nie hier.“",
      ],
    });
    await muehleEnde(rt, held);
    return;
  }
  await rt.present({
    held,
    probe: ergebnis,
    lines: [
      "Rennik lacht einmal, kurz und
      trocken.",
      "„Schulden haben hier jeder. Deine
      Stimme wiegt weniger als mein Getreide.“",
      "Der Wächter rückt näher. Reden allein
      reicht nicht.",
    ],
  });
  return;
}

const kampf = probe(held, "Stärke",
held.staerke, MITTEL, "Rennik aus dem Lagerhaus
treiben");

```

```

    if (kampf.erfolg) {
        held.loesungswegMuehle = "kampf";
        held.renniksBeweis = true;
        held.fluechtlingeEntdeckt = true;
        await rt.present({
            art: "combat",
            held,
            probe: kampf,
            lines: [
                "Der Tisch kippt. Die Waage geht zu
                Boden. Korn mischt sich mit Dreck.",
                "Rennik flieht, bevor der Wächter ganz
                liegt. An der Wand bleibt das Bündel
                Schuldscheine.",
                "Bertoks Zeichen liegt zuoberst. Du
                musst es nicht suchen, nur nehmen.",
            ],
        });
        await muehleEnde(rt, held);
        return;
    }
    const wunde = schaden(held, 4, "Kampf im
    Kontor");
    held.todesort = tot(held) ? "rennik" :
    held.todesort;
    await rt.present({
        art: tot(held) ? "death" : "combat",
        held,
        probe: kampf,
        log: [wunde],
        lines: tot(held)
            ? ["Rennik wiegt weiter, während du
            aufhörst, Gewicht zu haben."]
            : ["Du wankst. Rennik bleibt sitzen. Das
            Kontor gehört ihm noch."],
    });
}

```

```

async function muehleVerrat(rt: Runtime, held:
Held) {
  const wahl = await rt.present({
    title: "Rathaus",
    art: "townhall",
    portrait: "holm",
    held,
    lines: [
      "Holm hört zu, ohne die Feder
abzusetzen.",
      "„Eine versteckte Familie in der Mühle.
Und ein Händler am Ufer, der Getreide wiegt, das
nicht seines ist.“",
      "Er sieht dich an, als müsste er
entscheiden, ob Schutz eine Form von Verrat
ist.",
    ],
    choices: [
      "Die Familie nennen, damit die Mühle
Schutz bekommt",
      "Den Satz zurücknehmen",
    ],
  });
  if (wahl === 1) return;
  held.loesungswegMuehle = "verraten";
  await muehleEnde(rt, held);
}

```

```

async function muehleEnde(rt: Runtime, held:
Held) {
  if (held.loesungswegMuehle === "verraten") {
    await rt.present({
      title: "Sicheres Mehl, leere Blicke",
      art: "mill",
      portrait: "holm",
      held,

```

```

        lines: [
            "Die Wache holt die Familie noch vor  
Mittag. Yorwin geht, ohne das zerbrochene  
Werkzeug loszulassen.",
            "Die Mühle bekommt ihren Schutzbrief.  
Das Mehl kommt pünktlich.",
            "Bertok grüßt dich künftig mit dem Kopf,  
nicht mit der Hand.",
        ],
    });
    return;
}

if (held.loesungswegMuehle === "kampf") {
    await rt.present({
        title: "Mehl mit rauen Händen",
        art: "mill",
        portrait: null,
        held,
        lines: [
            "Das Mehl fließt wieder, aber im Dorf  
redet man über die Männer, die man am Steg  
gesehen hat, mit blutiger Nase.",
            "Bertok bedankt sich knapp und schließt  
die Tür einen Spalt früher als nötig.",
            "Lene zählt die Säcke nicht mehr. Sie  
steht vor der Nische, als könnte Zählen sie  
verraten.",
        ],
    });
    return;
}

const gold = goldPlus(held, 2, "ein Sack Mehl,  
der sich verkaufen lässt");
await rt.present({
    title: "Stilles Mehl",

```

```
    art: "mill",
    portrait: "miller",
    held,
    log: [gold],
    lines: [
        "Am nächsten Morgen dreht sich das Rad
lauter als sonst.",
        "Bertok reicht dir einen vollen Sack, ohne
ein Wort über die Nacht zu verlieren.",
        "Aus der Kornkammer ist nichts mehr zu
hören.",
        "Manche Schulden werden nicht bezahlt, nur
nicht mehr eingetrieben.",
    ],
    });
}
```